

Computación distribuida.

Profesor: Fernando Michel Tavera.

Ayudante de laboratorio: Luis Mario Escobar Rosales .

Sobre el desarrollo de la práctica:

La práctica resultó bastante difícil debido a nuestra inexperiencia utilizando C. En C a diferencia de Java, los apuntadores juegan un papel muy importante a la hora de modelar un problema. Esto es justamente lo que cuesta mucho comprender, osea, cómo utilizar apuntadores para lograr cosas que en Java no necesitan especificarse explícitamente. Lamentablemente en esta ocasión sólo pudieron “completarse” las estructuras Pila y Cola.

Respecto a las pruebas, no fueron demasiado complicadas y la instalación de las herramientas necesarias pudo llevarse a cabo correctamente.

Sobre la implementación:

Para la implementación de Pila y Cola se utiliza una estructura común llamada Nodo. Los nodos son los componentes esenciales de las pilas y colas, un nodo tiene un apuntador que guarda la dirección en memoria del nodo siguiente. El problema fue representar el elemento que se almacena en el nodo ya que C sólo tiene 5 tipos básicos. En un principio se trató de utilizar void*, osea un puntero sin tipo específico, pero resultó muy confuso en un principio y se optó por los nodos que almacenan sólo números enteros. De esta manera, nuestra implementación modela Pilas y Colas de números enteros.

En lo que refiere al caso de las estructuras Pila y Cola notará que son bastante parecidas. Las funciones de Pila y Cola permiten inicializarlas, agregar y eliminar elementos, entre otras cosas. Lo más destacable es que cuando la Pila está vacía y se agrega un elemento, el elemento agregado se establece como nuevo tope pero además su elemento siguiente apunta al antiguo tope. Para las colas, al agregar el primer elemento este se establece como primero y último. Cuando se agregan elementos después del primero se hace algo análogo a las pilas pero esta vez modificando quien es el último y el siguiente del antiguo último.

Una limitación de la implementación por ejemplo es que cuando la Pila o la cola es vacía y queremos sacar un elemento, se regresa -1 lo cual puede resultar confuso.

Pruebas de Rendimiento:

El alumno deberá utilizar el programa auxiliar cuyo código fuente está disponible en el Classroom del curso, como sigue:

Primero, compilar el archivo Ejercicio.c, ya sea utilizando la instrucción:

- mpicc Ejercicio.c -o ejercicio

Una vez compilado el código, lanzar el ejecutable resultante con diferentes cantidades de nodos. Por ejemplo:

Computación distribuida.

Profesor: Fernando Michel Tavera.

Ayudante de laboratorio: Luis Mario Escobar Rosales .

- `mpirun --oversubscribe -np 4 ./ejercicio`

para ejecutar el programa con 4 nodos.

La directiva `-np X` ejecuta el programa utilizando X nodos, mientras que la directiva `--oversubscribe` permite a MPI utilizar más nodos que el número de núcleos de procesamiento con los que cuenta el equipo.

El ejercicio consiste en realizar varias ejecuciones del programa con diferentes cantidades de nodos, tomando nota del tiempo de ejecución reportado, y utilizar esa información para calcular aproximaciones de las métricas.

Con los resultados obtenidos se deberá llenar la siguiente tabla:

Número de nodos	Tiempo de ejecución	Speedup	Eficiencia	Fracción Serial
n	T(n) μ s	S(n)	E(n)	F (n)
1	364454	1.0000	1.0000	_____
2	354503	1.0281	0.5140	0.9454
3	354422	1.0283	0.3428	0.9587
4	369891	0.9853	0.2463	1.0199
6	374934	0.9720	0.1620	1.0345
8	391155	0.9317	0.1165	1.0837
10	544620	0.6692	0.0669	1.5493
20	1166627	0.3124	0.0156	3.3169

1. Las métricas obtenidas serán solo aproximaciones, debido a que se calculan a partir del tiempo de ejecución en un experimento práctico y no con base en la complejidad del algoritmo.
2. A veces una ejecución en particular puede tomar más tiempo de lo normal debido a procesos de manejo del sistema operativo u otras circunstancias. Por ello, puede ser buena idea ejecutar varias veces con el mismo número de hilos para tener una mejor idea de cuánto es el tiempo “normal” y cuáles ejecuciones pueden considerarse anomalías.
3. En algunos sistemas, la ejecución puede tardar un tiempo excesivo cuando se usan demasiados nodos con la directiva `--oversubscribe`. Si este es el caso al realizar las pruebas por ejemplo, si una corrida tarda varios minutos—, pueden omitirse las pruebas con más

Computación distribuida.

Profesor: Fernando Michel Tavera.

Ayudante de laboratorio: Luis Mario Escobar Rosales .



nodos, siempre que se incluya en el reporte una explicación de por qué no se realizaron dichas pruebas.

4. Las fórmulas para calcular cada una de las métricas se incluyen en el archivo métricas.pdf.

5. Esta práctica es opcional, pero tiene un valor de hasta un punto adicional sobre la calificación de alguna práctica.

Métricas

Definiciones

$T(1)$: complejidad del mejor algoritmo secuencial conocido.

$T(p)$: complejidad del algoritmo paralelo usando p procesadores.

Aceleración (Speedup)

$$S(p) = \frac{T(1)}{T(p)}$$

Mide cuántas veces más rápido es el algoritmo paralelo. El valor ideal es p .

Eficiencia

$$E(p) = \frac{S(p)}{p}$$

Mide qué tan eficientemente se están utilizando los procesadores. El valor ideal es 1.

Fracción Serial

$$F(p) = \frac{\frac{1}{S(p)} - \frac{1}{p}}{1 - \frac{1}{p}}$$

Mide la parte del código que es inherentemente secuencial. El valor ideal es 0.